

GATE CS/IT: Computer Networks

Access Control

GATE Aspirant

July 23, 2026

What is Access Control (Media Access Control)?

- **The Core Problem:** So far, we looked at Flow Control (S&W, GBN, SR) which assumes a dedicated point-to-point link between one sender and one receiver. But what happens if multiple stations share a single **broadcast medium** (like a single Wi-Fi channel or a shared Ethernet cable)?
- **The Risk of Collision:** If two or more stations attempt to transmit data simultaneously on a shared link, their electrical/radio signals mix together and corrupt each other. This is called a **collision**.
- **Media Access Control (MAC):** A sublayer of the Data Link Layer responsible for coordinating access to the shared channel. It answers a simple question: *"When multiple stations want to talk, whose turn is it to transmit?"*

Taxonomy of Access Control Protocols

- Access Control protocols are broadly classified into three categories based on how they mediate channel access:
- **1. Random Access (Contention) Protocols:**
 - No station is superior to another. Stations compete freely for the channel. Collisions *will* happen, so the protocol must detect or avoid them.
 - *Examples:* Pure ALOHA, Slotted ALOHA, CSMA, CSMA/CD (Ethernet), CSMA/CA (Wi-Fi).
- **2. Controlled Access Protocols:**
 - Stations consult one another to find which station has the right to send. A station cannot send unless it has been authorized.
 - *Examples:* Token Passing (Token Ring), Polling, Reservation.
- **3. Channelization (Multiplexing) Protocols:**
 - The available bandwidth of the link is divided logically among different stations based on time, frequency, or coding.
 - *Examples:* TDMA (Time), FDMA (Frequency), CDMA (Code).

Time Division Multiplexing (TDM): The Concept

- **What is it?** TDM is a channelization protocol where the entire bandwidth of the shared medium is allocated to a single station, but only for a very short, dedicated period of time.
- **The Mechanism:**
 - Time is divided into discrete, equal-sized **Time Slots**.
 - These time slots are grouped into **Frames**.
 - Each station is assigned a specific slot within every frame to transmit its data (e.g., Station A gets Slot 1, Station B gets Slot 2, etc.).
- **Analogy:** Imagine a panel of 4 speakers sharing a single microphone. TDM is like giving the microphone to Speaker A for exactly 1 minute, then Speaker B for 1 minute, and so on, in a strict round-robin fashion.
- **Usage:** Widely used in traditional digital telephony (like the T1/E1 carrier systems).

Synchronous vs. Statistical TDM

• 1. Synchronous TDM:

- *How it works:* The time slot assignment is rigidly fixed. If there are N stations, the frame is always divided into exactly N slots.
- *The Flaw:* If a station has no data to send during its turn, its assigned time slot travels empty. This leads to massive bandwidth wastage in bursty network traffic.

• 2. Statistical (Asynchronous) TDM:

- *How it works:* Designed to fix the inefficiency of Synchronous TDM. Slots are allocated dynamically based on demand rather than being pre-assigned.
- *The Mechanism:* The multiplexer checks which stations actually have data ready to send and only allocates slots to those active stations.
- *Trade-off:* Because slots aren't fixed, each time slot must now carry a small **address header** identifying which station the data belongs to, which adds slight overhead but vastly improves overall channel utilization.

Introduction to CSMA/CD

- **What it stands for:** **C**arrier **S**ense **M**ultiple **A**ccess with **C**ollision **D**etection.
- **Where it is used:** It is the foundational access control protocol for traditional wired Ethernet (IEEE 802.3) over shared mediums (like coaxial cables or hubs).
- **The Motivation:** In basic CSMA, if two stations start transmitting at the same time, their frames collide. Basic CSMA stations will blindly transmit the entire frame, wasting massive amounts of bandwidth on corrupted data.
- **The CSMA/CD Improvement:** CSMA/CD adds a crucial capability: stations actively monitor the channel *while* they are transmitting. If they detect a collision, they abort the transmission immediately rather than finishing the useless frame.

How CSMA/CD Works: The 3-Step Process

- **Step 1: Carrier Sense (Listen before you talk)**

- Before sending data, a station checks the physical wire. If it senses electrical activity (someone else is talking), it waits.
- If the wire is idle, it begins transmitting.

- **Step 2: Multiple Access (Listen while you talk)**

- Multiple stations might have sensed an idle channel and started transmitting simultaneously.
- Therefore, while the station is sending its frame, it constantly measures the voltage levels on the wire.

- **Step 3: Collision Detection (Voltage Spikes)**

- If two signals collide on a copper wire, their electrical energies combine. The transmitting station detects a sudden voltage spike that exceeds normal transmission levels.
- This spike is the definitive proof of a collision.

Collision Resolution: The Jam Signal & Backoff

- **Aborting Transmission:** The moment a collision is detected, the station immediately stops sending the data frame to stop wasting bandwidth.
- **The Jam Signal:**
 - The station transmits a short, 32-bit dummy sequence called a "Jam Signal".
 - This ensures that the collision lasts long enough for every other station on the network to notice it and discard the corrupted fragments.
- **Binary Exponential Backoff Algorithm:**
 - After the Jam Signal, the station must wait before trying again.
 - To prevent the same stations from immediately colliding again, they wait for a **random** amount of time.
 - If collisions keep happening (heavy traffic), the range of the random waiting time doubles exponentially (e.g., choose a random time between 0-1, then 0-3, then 0-7...). This dynamically spaces out the traffic.

The CSMA/CD Golden Rule: $T_t \geq 2 \times T_p$

- **The Fatal Flaw:** For collision detection to work, a station must *still be transmitting* its frame when the collision signal bounces back to it. If a station finishes transmitting and drops the frame from its buffer before the collision signal arrives, it will falsely assume the frame was delivered successfully!
- **Defining the Variables:**
 - T_t (**Transmission Time**): The time it takes a station to push the entire frame onto the wire.
 - T_p (**Propagation Time**): The time it takes for one bit to travel from one end of the cable to the other.
- **The Worst-Case Scenario:** The most extreme delay in detecting a collision occurs when two stations are located at the absolute extreme ends of the network cable.
- **The Condition:** To guarantee a collision is detected under worst-case conditions, the transmission time must be strictly greater than or equal to the Round Trip Time (RTT).

$$T_t \geq 2 \times T_p$$

Example: The Worst-Case Collision Trace

- Imagine Station A and Station B at opposite ends of a long coaxial cable. Let the propagation delay $T_p = 10\mu s$.
- **Time $t = 0$:** Station A begins transmitting a frame.
- **Time $t = 9.99\mu s$:** The first bit of A's frame is just a millimeter away from Station B. Station B senses the wire. Because A's signal hasn't technically reached B yet, B thinks the channel is idle!
- **Time $t = 10\mu s$ (T_p):** Station B begins its own transmission.
COLLISION occurs instantly at B's end of the wire.
- **The Return Journey:** Station B detects the collision immediately, but Station A is completely unaware. The high-voltage collision spike must travel all the way back across the cable to reach A.
- **Time $t = 20\mu s$ ($2 \times T_p$):** The collision signal finally reaches Station A.
- *The Catch:* If Station A's frame was so small that it only took $15\mu s$ to transmit ($T_t = 15$), A would have finished and stopped listening $5\mu s$ ago. It would miss the collision! Therefore, A's transmission **must last at least $20\mu s$.**

Deriving Minimum Frame Size

- Network engineers use the $T_t \geq 2 \times T_p$ rule to calculate physical limits for network hardware.
- We know that:
 - Transmission Time (T_t) = $\frac{\text{Length of Frame } (L)}{\text{Bandwidth } (B)}$
- Substituting this into the collision condition:

$$\frac{L}{B} \geq 2 \times T_p$$

- Rearranging for L , we get the **Minimum Frame Size Equation**:

$$L \geq 2 \times T_p \times B$$

- **Real-World Impact:** This exact equation is why classic Ethernet (IEEE 802.3) enforces a strict **minimum frame size of 64 bytes**. If a computer wants to send only 10 bytes of data, it must add 54 bytes of useless "padding" to ensure the frame is long enough to survive a worst-case collision!

Efficiency of CSMA/CD: The Mathematical Model

- **The Efficiency Formula:** The theoretical maximum efficiency (or channel utilization) of CSMA/CD is given by the formula:

$$\eta = \frac{1}{1 + c \cdot a}$$

- **Understanding the Variables:**

- $a = \frac{T_p}{T_t}$: The ratio of Propagation Delay to Transmission Time.
- c : A constant representing the contention overhead. In standard CSMA/CD models, c is derived from the probability of a successful transmission after collisions and is generally taken as 6.44 (sometimes approximated as 6.4 or 5.44 depending on the specific probabilistic model).

- **The Simplified Equation:** For most standard engineering calculations and GATE problems, the efficiency is written as:

$$\eta = \frac{1}{1 + 6.44a}$$

Factors Influencing CSMA/CD Efficiency

- To maximize efficiency ($\eta \rightarrow 1$), we must make the parameter a as small as possible. Let's break down a :

$$a = \frac{T_p}{T_t} = \frac{\frac{\text{Distance } (d)}{\text{Velocity } (v)}}{\frac{\text{Frame Length } (L)}{\text{Bandwidth } (B)}} = \frac{d \times B}{v \times L}$$

- Based on this expansion, CSMA/CD efficiency is affected by:
 - ① **Network Length (d):** Inversely proportional. Shorter cables = higher efficiency.
 - ② **Bandwidth (B):** Inversely proportional. Higher data rates (like Gigabit Ethernet) drastically *reduce* CSMA/CD efficiency.
 - ③ **Frame Size (L):** Directly proportional. Larger packets mean the station spends more time transmitting useful data rather than contending for the channel.

Why CSMA/CD is Obsolete in Modern Networks

- **The High-Speed Bottleneck:** Look at the relationship $a \propto \text{Bandwidth} \times \text{Distance}$.
- As networks evolved from 10 Mbps to 1 Gbps (a 100x increase in bandwidth), the parameter a exploded, driving the efficiency of CSMA/CD close to zero for standard distances.
- **The Solution (Modern Ethernet):**
 - To maintain efficiency at high speeds, networks had to either shrink the cable distance to just a few meters (impractical), or mandate enormous minimum frame sizes.
 - Instead, modern networking abandoned shared channels entirely. Today's Ethernet uses **Switches** with dedicated, **Full-Duplex** links.
 - Because a full-duplex link has separate send and receive wires, **collisions are physically impossible**. Therefore, CSMA/CD is completely disabled in modern Switched Ethernet!

The Need for a Backoff Algorithm

- **The Problem:** In CSMA/CD, if Station A and Station B collide, they both send a Jam Signal and abort. If they both wait the exact same amount of time before retrying, they will just collide again.
- **The Solution: Binary Exponential Backoff (BEB).** It forces stations to wait for a *random* amount of time before retransmitting.
- **Dynamic Adaptation:** If traffic is light, the random wait time should be short. If the network is congested (indicated by repeated collisions), the stations should wait much longer to clear the traffic jam. BEB handles this dynamically.

How Binary Exponential Backoff Works

- **Step 1: Track Collisions.** Each station maintains a collision counter, let's call it c . For a new frame, $c = 0$. After a collision, c increments by 1.
- **Step 2: Determine the Backoff Range.**
 - The station calculates a parameter k , where $k = \min(c, 10)$.
 - It then defines a collision window range from 0 to $2^k - 1$.
- **Step 3: Pick a Random Slot.** The station chooses a random integer R from that range:

$$R \in [0, 2^k - 1]$$

- **Step 4: Calculate Wait Time.** The station waits for $R \times \text{Slot Time}$.
 - **Slot Time** is equal to the worst-case Round Trip Time ($2 \times T_p$). In classic 10 Mbps Ethernet, 1 Slot Time = $51.2 \mu\text{s}$ (the time to send 64 bytes).

The "Exponential" and "Truncated" Behavior

- **Why "Exponential"?** Notice how the range doubles after every failure:
 - 1st collision ($c = 1$): $k = 1$. Range is $[0, 1]$. (Station waits 0 or 1 slots).
 - 2nd collision ($c = 2$): $k = 2$. Range is $[0, 3]$.
 - 3rd collision ($c = 3$): $k = 3$. Range is $[0, 7]$.
 - 4th collision ($c = 4$): $k = 4$. Range is $[0, 15]$.
- **The Truncation ($k \leq 10$):** To prevent waiting delays from becoming absurdly long, the maximum value for k is capped at 10. For collisions 10 through 15, the range remains frozen at $[0, 1023]$.
- **The Abortion Limit ($c = 16$):** If a station attempts to send the same frame 16 times and experiences 16 consecutive collisions, the algorithm gives up. The frame is dropped, and an error is reported to the upper layers (Hardware failure or extreme network overload).

The Capture Effect in CSMA/CD

- **What is it?** The Capture Effect is a notorious fairness flaw inherent in the Binary Exponential Backoff (BEB) algorithm used by CSMA/CD.
- **The Definition:** It is a phenomenon where a single station, by pure statistical luck, monopolizes the shared network medium. It continuously "captures" the channel, forcing other waiting stations into longer and longer backoff periods.
- **The Root Cause:** BEB favors the station that has most recently transmitted a frame successfully, because successful transmission resets a station's collision counter back to zero.

How Capture Happens: A Step-by-Step Scenario

- **The Setup:** Station A and Station B collide. Their collision counters are $c_A = 1$, $c_B = 1$. Both choose a random backoff from $[0, 1]$.
- **Round 1:** Assume A picks 0 and B picks 1. Station A wins, transmits its frame successfully, and resets its counter to $c_A = 0$.
- **Round 2:** Station A immediately has a *new* frame to send. Station B finishes its wait time and tries to send its *old* frame. They collide again!
- **The Mismatch:**
 - For Station A, this is the 1st collision for its new frame: $c_A = 1$, range is $[0, 1]$.
 - For Station B, this is the 2nd collision for its old frame: $c_B = 2$, range is $[0, 3]$.
- **The Result:** Station A has a much higher probability of picking a shorter wait time and winning the channel again.

Consequences: Starvation vs. Efficiency

- **The Starvation Spiral:** If A has a massive file to transfer, this cycle repeats. A's range stays at $[0, 1]$, while B's range explodes to $[0, 7], [0, 15] \dots [0, 1023]$. Station B becomes completely "starved" and might eventually drop the frame entirely (reaching 16 collisions).
- **The Ironic Trade-off:**
 - **Fairness:** The Capture Effect is terrible for fairness. Latency for Station B skyrockets.
 - **Efficiency/Throughput:** Surprisingly, it is *great* for overall channel efficiency! Because Station A dominates, the channel spends less time dealing with collisions and backoffs, and more time successfully transmitting A's data.
- *Modern Context:* Just like collisions themselves, the Capture Effect is a historical artifact of shared Ethernet (hubs/coaxial) and does not exist in modern switched, full-duplex Ethernet.

Introduction to ALOHA

- **What is it?** ALOHA is one of the earliest Random Access protocols for Media Access Control (MAC), developed in the 1970s at the University of Hawaii for wireless satellite communication.
- **The Core Philosophy:** Extreme simplicity. Unlike CSMA, stations do *not* listen to the channel before transmitting (No Carrier Sense).
- **The Rule:** *"If you have data to send, send it immediately."*
- **Collision Handling:** Because stations transmit blindly, collisions are frequent.
 - After transmitting a frame, the sender waits for an Acknowledgment (ACK) from the receiver.
 - If no ACK arrives within a specific timeout period, the sender assumes the frame was destroyed in a collision.
 - The station then waits a random backoff time and retransmits the frame.

Pure ALOHA and the Vulnerable Time

- **Pure ALOHA:** The original version where a station can begin transmitting at any absolute millisecond.
- **The Vulnerable Time (V_t):** The window of time during which a transmission can be corrupted if any other station also decides to transmit.
- Let T_t be the time it takes to transmit one frame.
- For a frame starting at time t , it will suffer a partial or full collision if another station starts transmitting anywhere between $t - T_t$ and $t + T_t$.
- **Vulnerable Time Equation:**

$$V_t = 2 \times T_t$$

- **Maximum Efficiency:** Due to this large vulnerable window, the maximum theoretical channel utilization is painfully low:

$$\eta_{\text{pure}} = \frac{1}{2e} \approx 18.4\%$$

Slotted ALOHA: The Improvement

- **The Goal:** Reduce the vulnerable time to improve overall network efficiency.
- **The Mechanism:**
 - Time on the channel is divided into discrete, synchronized intervals called **Slots** (where 1 Slot exactly equals T_t).
 - **The New Rule:** A station cannot transmit the exact moment data arrives. It must buffer the data and wait for the beginning of the *next* time slot.
- **The Result:** By forcing all transmissions to align exactly with slot boundaries, partial frame overlaps are eliminated. Frames now either collide completely or not at all.
- **Vulnerable Time Equation:**

$$V_t = 1 \times T_t$$

- **Maximum Efficiency:** Cutting the vulnerable time in half exactly doubles the maximum theoretical efficiency:

$$\eta_{\text{slotted}} = \frac{1}{e} \approx 36.8\%$$